

# Optimizing Dynamic Job Shop Scheduling with Greedy and Genetic Algorithms

Jacob Benny Packiaraj  
Master of Science- Data Analytics  
National College of Ireland  
Dublin, Ireland

**Abstract**—The permutation flow job shop scheduling is a popular optimization problem which is utilized in many manufacturing facilities. Because of the permutations involved, solving the problem was thought to be static, however, in several practical cases, the problem is considered dynamic where the job must schedule in a given number of machines in a cyclical order. To minimize the makespan of the overall flow, several algorithms were considered to solve this NP-hard problem and recent approaches uses a few iterative and heuristic approach to find ideal sequence of job schedule that minimize the overall execution time of the machines. In this paper, the usage of Greedy, Iterated Greedy, and Genetic Algorithms was used to find the near optimal solution to minimize the total execution time and maximize the utilization of the given number of machines. The performance of the algorithm is tested and the makespan calculated with this is compared with the benchmark solution obtained from integer programming.

**Keywords**—Greedy Algorithm, Genetic Algorithm, PFSP, Makespan, Heuristics.

## I. INTRODUCTION

The Permutation flow shop scheduling problem (PFSP) is a classic problem in computational programming and operations research where given a set of  $N=\{1, \dots, n\}$  of  $n$  jobs that have to be processed by a set of  $M=\{1, \dots, m\}$  of  $m$  machines. In this PFSP all jobs visit the machines in the same order which in this case is  $1, 2, 3, \dots, m$ . Total number of operations is  $n \times m$ .  $\pi = (\pi_1, \dots, \pi_n)$  be the job permutation, then makespan  $C(\pi_i, j)$  is calculated as  $C_{max}(\pi) = C(\pi_n, m)$ . The objective is to find a permutation  $\pi^*$  that minimizes  $C_{max}(\pi)$ . This problem is considered a NP-hard optimization, Nondeterministic Polynomial class, where it is least as hard as the hardest of other problems in NP.

Recent advancement in solving this problem involves the use of advanced heuristic techniques like Genetic Algorithm, Ant colony optimization to achieve the ideal job sequence for solving the job shop scheduling problem.

Recent researchers in the field of Greedy algorithm [1] where the use of an iterative approach to solve in minimizing the execution time for the permutation based flow shop problem. The study includes a group study for iteratively generate the solution sequence from greedy algorithm. It also utilize a destruction phase which is a good addition in increasing the span of solution set. Then it implements a local search from this solution set to achieve the best optimal or near optimal solution for the jobs to be scheduled. Few drawbacks in this approach is the utilization of group thinking which does not have a scope for solving extension problems.

The research conducted in [2] implements an iterated greedy approach keeping the objective of makespan to find the optimal solution. This work is similar in implementation to the research in [1], where it applies local search to the partial

solution after the destruction phase. The iterated greedy algorithm implemented in this project by this author will use a local search to the solutions obtained on the partial solutions obtained for finding an optimal value.

In research conducted in [3], consists of a comprehensive study on the iterated approaches in Greedy algorithm over the years covering 100 papers. The basic implementation of the Iterated greedy approach is taken by the author to use in this project. Different strategies is explained by the researchers on the variation of the iterative strategies however most approach uses single solution strategy which do not give the best optimal solution.

Genetic Algorithm is another approach where the optimal solution is obtained by natural selection and having the highest fitness value. For solving the PFSP, [4] implement a Genetic Algorithm focusing predominantly on the total flowtime which is the makespan of all the jobs scheduled on each machine. Population initialization and other operators of selection, crossover and mutation is implemented followed by a local search. Basic implementation of the algorithm is carried out in this research work and this is considered as a baseline method by this author in the research work.

A detailed study of the application of the Genetic Algorithm on the make-to-stock production system is carried out [5], uses a cost function to reduce the operating cost of those machines associated with the jobs with all other constraints of the job shop scheduling process. Memetic algorithm, a metaheuristic approach is incorporated for the complex optimization for the sanitaryware production systems. It is a metaheuristic approach in identifying the best solution set from the overall population.

Interesting work is carried out in scheduling problem research in [6] using a self-guided Genetic Algorithm implementation, using a probabilistic model for performing crossover and mutation. Here the self-guided crossover operator using the two-point centre crossover provided results which are statistically significant comparing to other Estimation of Distribution Algorithms which is set as a benchmark to find the performance of the Genetic approach.

From the survey conducted, an iterative Greedy algorithm along with the basic implementation of Greedy algorithm is implemented in the study. The iterative method uses a local search criteria to optimize the solution obtained and another Genetic Algorithm with a two-point crossover to find the fittest solution is implemented to find the performance of algorithms.

## II. ALGORITHMS

The input data is generated using a generateData function, where based on the number of machines and number of jobs to be scheduled on each machine, a sequence of integers are

randomly generated in a 2-D array where each array represents the amount of time units each machine takes to process the job. In this implementation of the algorithm, the problem tried to address the Permutation Flow shop Scheduling Problem (PFSP) where given 'n' number of jobs and 'm' number of machines, the jobs are to be scheduled in each machines following a similar order for the group of jobs scheduled to be completed.

#### A. Greedy Algorithm

Greedy algorithm is a technique used to find an ideal schedule for a given set of jobs in an order that reduces the overall execution time for all the jobs to complete. Based on certain heuristics, the sequence of the jobs are identified to schedule. In this case, jobs with shortest evaluation time is identified and does a local optimal choice to reduce the makespan in each step to identify the near optimal sequence of jobs.

The algorithm is implemented with a simple greedy approach where the result aims to find the order for scheduling jobs on the given number of machines such that the jobs are tagged to the machine in a way that minimize the makespan of the complete set of jobs on each machine. The jobs are sorted based on the minimum processing time on all the machines. The logic tries to find the machine that results in the earliest completion time for the given job from the sorted list. The function returns a sequence of jobs, where the sequence are assigned in a way which aims to minimize the makespan by making local optimal choice of the job completion time on each step.

#### B. Iterated Greedy Algorithm

Another iterative based greedy search is implemented in a function which aims to minimize the maximum completion time by iteratively improving the sequence of the jobs. Here the local search for optimization, iterating multiple times to provide the best optimal sequence of jobs efficiently. Elapsed time to reach the optimal sequence is calculated to perform the comparison with other algorithm processing time.

The final sequence and the 2D array is passed to another plot\_gantt\_chart function which iterates through the jobs and their correspondingly assigned machines and plot the execution time of each jobs.

A function to calculate the idle time is implemented on each machine, where given the job execution time and the optimal sequence, calculate the idle time of each machine to evaluate the performance of the scheduling algorithm.

#### C. Genetic Algorithm

Another algorithm technique used in this project is to find optimal solution by using the principles of natural selection and genetics using Genetic Algorithm. In particular to the job shop scheduling problem (JSSP), the overall goal is to optimize and schedule a set of jobs on the given number of machines such that the makespan is minimized similar to greedy algorithm.

This approach utilizes the implementation of the algorithm where based on the population size, crossover rate, mutation rate, mutation selection rate and number of iterations to be carried out generates a solution closer to the optimal result.

The key operators in Genetic Algorithm includes the crossover, repairing offspring, mutation, fitness evaluation

and selection. The crossover utilized is the two-point crossover, where for each pair of parents based on certain rate, two points are selected in random and the segments are swapped to create another offspring.

This followed by repairing offspring which ensures each job appears the expected number of times by adjusting any job counts that are high or low. Mutation is the process where the subset of jobs in a offspring is chosen and based on the mutation rate fixed, the order is shuffled a bit to increase the fitness value of the given chromosome.

Fitness value is another operator which picks the solution having the highest value after converting to the decimal equivalent making sure all the machines are running to get to the near optimal solution to schedule the jobs to reduce the makespan. The optimal or the closest sequence is the best solution which in GA is the best chromosome that is identified from the population.

For the chromosome that gives the highest fitness score is used to calculate the makespan, which is the total time taken to process all the set of jobs on each machine. The resultant sequence is visualized using a Gantt chart to check the scheduling of the jobs and its sequence. Idle time of each machine is calculated to find the effectiveness of the algorithm.

#### D. Performance Measurement

The results obtained from the above algorithms are evaluated with the solutions obtained from Integer programming solved using Pulp package in python.

Algorithms are tested with 'number of machines' as 4 and 'number of jobs' as 100 and the performance is evaluated using the elapsed time to find the best solution and the total job time to be processed by all machines.

From table 1, The integer solution obtained using pulp library took 104 seconds to identify the sequence, whereas the iterated Greedy algorithm approximately took 153 seconds with the iteration of 200. Similarly in Genetic Algorithm, with a fixed population size of 30, crossover rate, mutation rate and mutation selection rate of 0.8, 0.2 and 0.2 respectively the number of iterations is varied with 2000, 1500 and 1000 runs. The execution time for each of these iterations are 129, 188 and 128 seconds approximately. However, the makespan obtained from the algorithm is optimal from the integer solution where it is 537 units. Greedy and Iterated Greedy algorithm provided a same sequence of jobs and has the makespan of 567 units. In Genetic Algorithm, 885 units, 862 units and 854 units were the calculated makespan obtained. Looking at the results, the Iterated greedy algorithm which is implemented provides a near to optimal solution obtained from the benchmark from Integer programming.

TABLE I. PERFORMANCE COMPARISON OF ALGORITHMS

| Algorithms                     | Evaluation Metrics |                  |            |
|--------------------------------|--------------------|------------------|------------|
|                                | Elapsed time (sec) | Makespan (units) | Iterations |
| Integer Programming            | 104                | 537              | -          |
| Greedy                         | -                  | 567              | -          |
| Iterated Greedy                | 153                | 567              | 200        |
| Genetic (Least execution time) | 128                | 854              | 1000       |

### III. EVALUATION

To closely evaluate the performance of the algorithm and how the jobs are scheduled in each machine, further test is conducted with fewer jobs to visually interpret how the machine are utilized and whether the algorithms used provide a near optimal solution to the benchmark values obtained from integer programming solution.

The test is conducted with number of jobs as 14 and machine counts as 4 for all the algorithms. Using the integer programming, the benchmark makespan is calculated to be 94 units and the idle time on each machine is 3 units in machine-1, 6 units in machine-2, 21 units in machine-3 and 10 units in machine-4.

#### A. Greedy and Iterated Greedy Algorithm

In this approach of Greedy and Iterated Greedy algorithm, using the implemented technique, the 14 jobs are scheduled on the 4 machines and the results are evaluated.

Here the sequence obtained from the Greedy approach is 1, 8, 11, 2, 4, 5, 6, 10, 3, 7, 13, 0, 12, 9 and the sequence obtained from the Iterated greedy search using a local optimization provide the sequence of 2, 8, 11, 1, 4, 5, 6, 10, 3, 7, 13, 0, 12, 9.

effectively the jobs are scheduled so that all the machines are utilized at a given time.

From Fig. 1, the idle time is calculated to be 0 units for machine-1, 15 units for machine-2, 18 units for machine-3 and 27 units for machine-4.

Similarly from Fig. 2, the idle time calculated is 0 units for machine-1, 9 units for machine-2, 17 units for machine-3 and 29 units for machine-4.

However the benchmark solution obtained from the Integer programming gives the optimal sequence of 7, 10, 8, 9, 2, 3, 1, 6, 13, 12, 0, 5, 11, 4 which has the objective or the makespan of just 94 units. The Greedy and Iterated Greedy provides a larger makespan of 111 and 113 units which shows the solution obtained from these algorithms are far from the optimal solution.

Increasing the iteration for the Iterated Greedy algorithm did not provide any significant improvement and provided the same solution sequence in all the iterations carried out.

One significant improvement in implementing the local optimization in the iterated greedy is that the idle time in machine-2 and machine-3 is reduced to some extent.

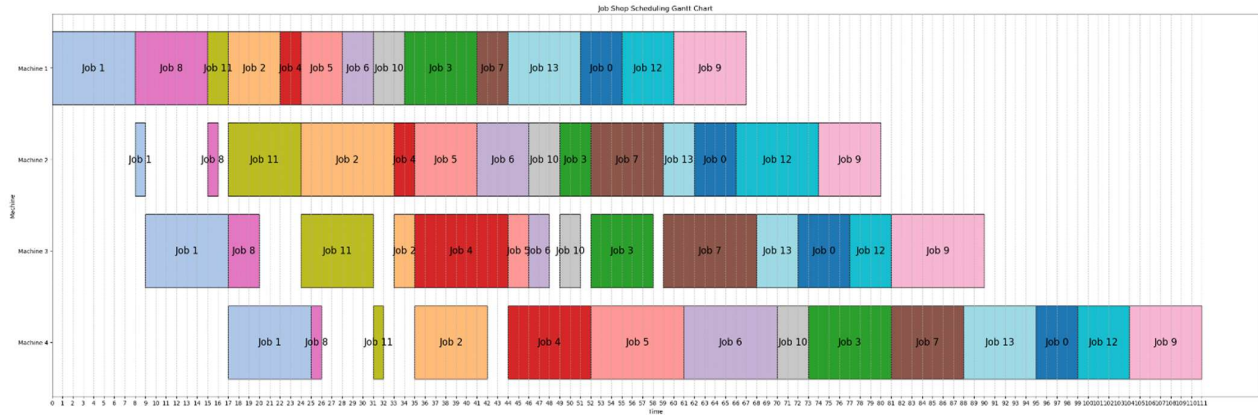


Fig. 1. Gantt Chart for Greedy Algorithm. ( $m=4$ ,  $n=14$ )

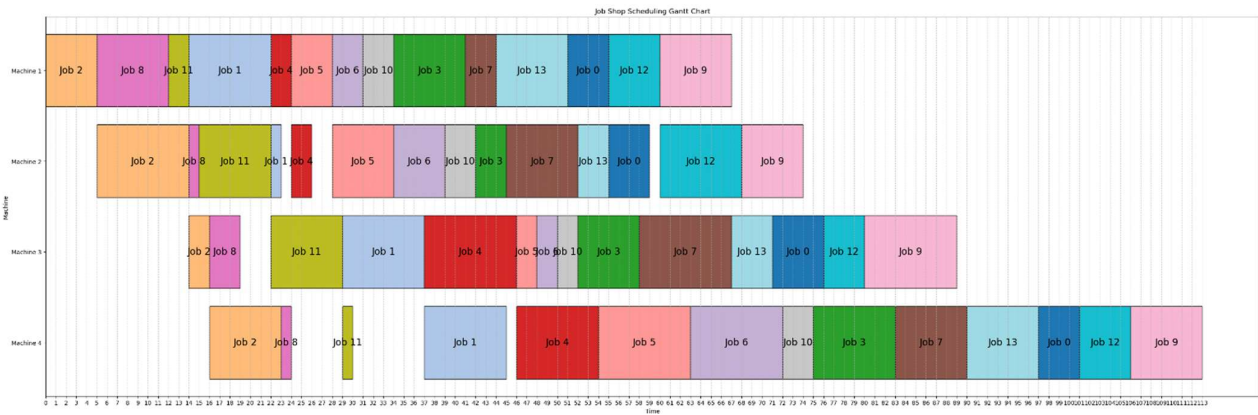


Fig. 2. Gantt chart for Iterated Greedy Algorithm. ( $m=4$ ,  $n=14$ )

The makespan calculated based on the above two sequences are 111 units for Greedy approach and 113 units for the Iterated Greedy approach. However, checking the Idle time of each machine gives broader insights on how

However, in machine-4, the idle time is quite high which causes the increase in the makespan of the entire job runs on all the machines.

### B. Genetic Algorithm

From the algorithm implemented using Genetic technique, similar test is conducted to visualize on how the jobs are scheduled.

Using the parameters in operating the Genetic algorithm, for a population size of 30, crossover rate, mutation rate, mutation selection rate of 0.8, 0.2 and 0.2 respectively with an iteration of 2000 generated a sequence where the makespan of 105 units.

The sequence of jobs to be executed by the machine is 5, 3, 3, 5, 11, 3, 3, 4, 5, 0, 4, 11, 7, 13, 4, 6, 7, 6, 13, 10, 4, 3, 7, 13, 9, 0, 6, 9, 2, 11, 8, 13, 9, 8, 1, 1, 6, 10, 9, 2, 7, 1, 8, 10, 12, 2, 8, 10, 0, 2, 1, 11, 12, 0, 12, 12. This represents the successive job on each machine which is picked by the algorithm to be executed. This is the mapping of each job to the corresponding machine as the implemented genetic algorithm do not provide any overall fixed order of jobs. This is due to the fact that, the algorithm tries to minimize the makespan and derive a solution that is best for the objective rather than a straightforward approach of solving a problem.

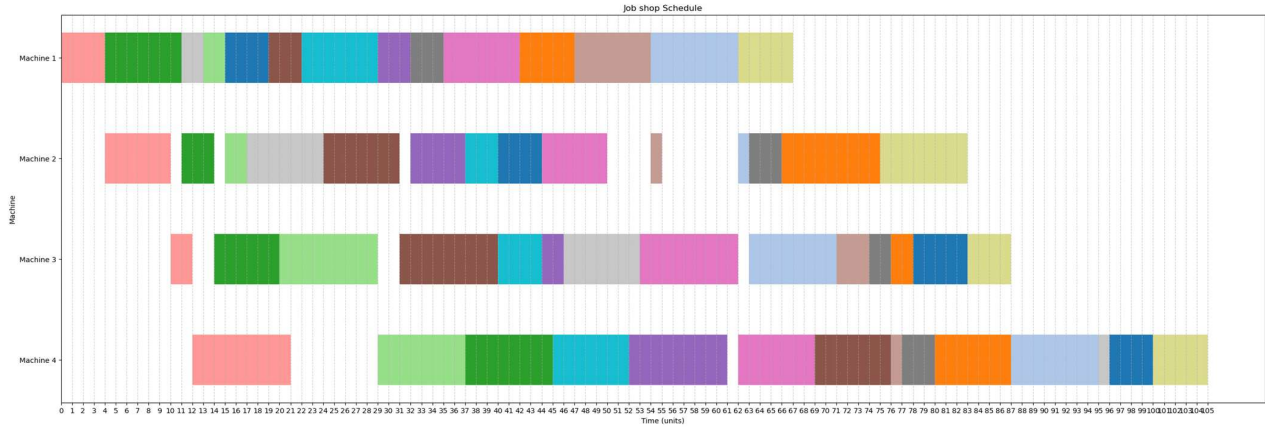


Fig. 3. Gantt chart for Genetic Algorithm. ( $m=4$ ,  $n=14$ )

The heuristic nature of the Genetic Algorithm tends to provide a near optimal solution in this scenario, where the makespan is closer to the optimal solution obtained from the Integer programming.

### C. Summary

From the best solutions obtained from each of the algorithms used, from table. 2, the optimal solution and the best solution is from the benchmark Integer programming where a makespan of 94 units is obtained. From the implemented algorithms between Greedy and Genetic Algorithms, Genetic approach provided a closer to optimal solution in this case where the total time required for the job completion is 105 units which is then followed by the Greedy and Iterated Greedy approach.

TABLE II. MACHINE IDLE TIMES FROM OTHER ALGORITHMS

| Algorithms          | Evaluation Metrics |         |         |         |         |
|---------------------|--------------------|---------|---------|---------|---------|
|                     | Makespan (Units)   | M1-Idle | M2-Idle | M3-Idle | M4-Idle |
| Integer Programming | 94                 | 3       | 6       | 21      | 10      |
| Greedy              | 111                | 0       | 15      | 18      | 27      |
| Iterated Greedy     | 113                | 0       | 9       | 17      | 29      |
| Genetic             | 105                | 0       | 18      | 15      | 21      |

Further improvement can be brought to the algorithms by changing the mutation rate, crossover rate, number of iterations and population of the chromosomes to be selected for the problem. Increasing the rate of mutation can help to reduce the makespan where the changes effected on the identified chromosome can produce a higher fitness score compared to the overall population, thereby producing a near optimal or optimal solution for the job sequence.

Using other metaheuristic techniques like Simulated Annealing(SA), Tabu Search, Partial Swarm Optimization are alternate algorithms for solving similar job shop scheduling problem.

### REFERENCES

- [1] Huang, Y.Y., Pan, Q.K., Huang, J.P., Suganthan, P.N. and Gao, L., 2021. An improved iterated greedy algorithm for the distributed assembly permutation flowshop scheduling problem. *Computers & Industrial Engineering*, 152, p.107021.
- [2] Dubois-Lacoste, J., Pagnozzi, F. and Stützle, T., 2017. An iterated greedy algorithm with optimization of partial solutions for the makespan permutation flowshop problem. *Computers & Operations Research*, 81, pp.160-166.
- [3] Zhao, Z., Zhou, M. and Liu, S., 2021. Iterated greedy algorithms for flow-shop scheduling problems: A tutorial. *IEEE Transactions on Automation Science and Engineering*, 19(3), pp.1941-1959.
- [4] Duan, J.H., Zhang, M., Qiao, G.Y. and Li, J.Q., 2011, May. A genetic algorithm for permutation flowshop scheduling with total flowtime criterion. In 2011 Chinese Control and Decision Conference (CCDC) (pp. 1514-1517). IEEE.
- [5] Rahman, H.F., Sarker, R. and Essam, D., 2015. A genetic algorithm for permutation flow shop scheduling under make to stock production system. *Computers & Industrial Engineering*, 90, pp.12-24.
- [6] Chen, S.H., Chang, P.C., Cheng, T.C.E. and Zhang, Q., 2012. A self-guided genetic algorithm for permutation flowshop scheduling problems. *Computers & operations research*, 39(7), pp.1450-1457
- [7] Logic for Genetic Algorithm is sourced from [https://github.com/bharatpurohit97/Job-Shop-Scheduling-GeneticAlgorithm/blob/master/Schedule\\_Genetic\\_Algorithm.ipynb](https://github.com/bharatpurohit97/Job-Shop-Scheduling-GeneticAlgorithm/blob/master/Schedule_Genetic_Algorithm.ipynb)